COSC 461/561 Project 1: Eagle Delivery

Arabi Rahman Rafi · https://github.com/arabirahmanrafi-svg/EagleDelivery

Part A: Planning a delivery with A*

Setting up the problem. *State:* where the robot is, written as (row, column). It can be on any walkway or building entrance. *Start state:* the cell of the starting building, for example S = (1, 1). *Actions:* move UP, DOWN, LEFT, or RIGHT. *Transition model:* a move takes the robot one cell in that direction. If that cell is a wall (#) or off the map, the robot stays where it is. *Goal test:* the robot is on the destination building; passing other buildings does not count. *Path cost:* the number of moves, where each move costs 1. A move into a wall wastes a step and goes nowhere, so the search ignores those moves.

Why BFS finds the shortest route. BFS first checks every cell 1 move away, then every cell 2 moves away, and so on. Every move costs the same (1), so the first time BFS reaches the goal, it has used the fewest moves possible.

Why Manhattan distance is admissible. The heuristic $h(s) = |r_s - r_g| + |c_s - c_g|$ counts how many rows and columns the robot still has to cross. Each move changes the row or the column by only one, so the robot needs at least that many moves even if there were no walls. Walls can only make the trip longer. So h never guesses more than the real distance, which is what admissible means. It is also consistent: one move changes h by at most 1.

How my A* works. A* keeps a priority queue (a heap) of cells to visit, ordered by f = g + h, where g is the number of moves so far and h is the Manhattan guess. Each entry is (f, order added, g, cell), so when two cells have the same f, the one added first comes out first. For every cell I store the best g found so far and its parent (the cell it came from). If I find a shorter way to a cell, I update both and add the cell to the queue again. When an old, out-of-date entry comes out of the queue, I skip it. I check for the goal when a cell comes out of the queue, and I count a cell as expanded when I look at its neighbors, the same way the BFS code does.

Map	Delivery task	BFS length	A* length	BFS expanded	A* expanded
Baseline	S-W	18	18	67	22
Baseline	L-D	22	22	79	62
Baseline	H-P	16	16	41	16
Detour	S-W	26	26	70	51
Detour	L-D	22	22	65	60
Detour	H-P	16	16	41	16

Table 1. BFS vs A* on the six required tasks (results/search_results.txt).

What the table shows. BFS and A* always find routes of the same length, because both always find a shortest route. A* expanded fewer cells in all six tasks, but how much it saves changes a lot. A* saves the most when the Manhattan guess is close to the real distance. For baseline S-W, the guess from S is 14 and the real route is 18, so A* expanded only 22 cells compared with 67 for BFS. H-P is similar (16 vs 41). For L-D, the guess is 14 but the real route is 22, because walls force a big detour. A* then has to check many cells that look promising but are not on the route, so it saves less (62 vs 79). In the detour map, the closed walkway at (3, 6) turns a corridor that points toward W into a dead end. A* goes down that corridor first, finds it blocked, and backs out, so its count rises from 22 to 51, while BFS barely changes (67 to 70). Detour L-D is the closest case (60 vs 65). So A* is not always much better: when the guess is misleading it wastes work, and in some maps it could even expand more cells than BFS.

Part B: Learning a delivery policy with Q-learning

How my code works. `epsilon_greedy` picks a random action with probability ε (using `rng.random()` and `rng.choice()`). Otherwise it picks the action with the highest Q-value, where actions never tried count as 0. If several actions tie, it picks one of them at random. `q_learning_update` first works out a target. At the goal, the target is just the reward. Otherwise it is the reward plus γ times the best Q-value at the next cell. Then it moves Q(s, a) toward that target by a step of size α: $Q(s,a) \leftarrow Q(s,a) + \alpha(\text{target} - Q(s,a))$. All 14 tests pass. **Settings:** 2,000 training episodes, 100 test episodes, at most 200 moves per episode, α = 0.3, γ = 0.95, ε = 0.2, no slipping, and seeds 0, 1, 2, each starting with an empty Q-table.

Task	Shortest route	Test success	Test moves (return)	First goal reached (episode)	Success rate, episodes 1-100	Episodes that hit the 200-move limit	Training moves, last 500 episodes
S-W	18	100%	18 (22)	3.0 ± 2.6	0.83 ± 0.01	17.0 ± 1.0	22.5 ± 0.1
L-D	22	100%	22 (18)	3.7 ± 0.6	0.91 ± 0.01	8.7 ± 0.6	27.7 ± 0.1
H-P	16	100%	16 (24)	2.0 ± 0.0	0.85 ± 0.00	15.3 ± 0.6	20.1 ± 0.0

Table 2. Baseline Q-learning on the campus map. Each value is the average ± standard deviation over seeds 0, 1, 2.

What I found. All nine runs (3 tasks × 3 seeds) delivered the package 100% of the time in testing, always on the shortest route (18, 22, and 16 moves). So the test returns were 40 - L: 22, 18, and 24, the same for every seed. Training happened in three stages. (1) *Exploring:* at the start the robot wanders a lot. Many early episodes hit the 200-move limit (8 to 18 per run, all before episode 150), but it still reached the goal within its first 6 episodes. All Q-values start at 0 and every move costs -1, so any action the robot has already tried looks worse than one it has not tried yet. That pushes it to try new paths. (2) *Learning:* the number of moves per episode drops fast (Figure 1). It levels off at about episode 60 for H-P, 120 to 140 for S-W, and 140 for L-D. L-D is slowest because its route is the longest, so the goal reward takes longer to spread back to the start. After episode 150, every training episode reached the goal. (3) *Leveling off:* training episodes stay a bit longer than the shortest route (22.5 vs 18, 27.7 vs 22, 20.1 vs 16), because with ε = 0.2 about 1 in 5 moves is still random. A longer route has more chances for a random wrong move, so L-D has the biggest gap.

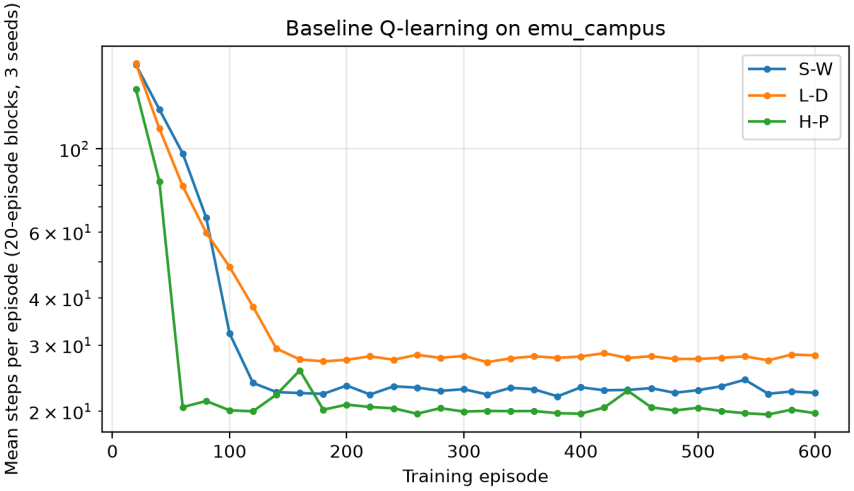


Figure 1. Average moves per training episode, in groups of 20 episodes (3 seeds, log scale), for the first 600 of 2,000 episodes.

Part C: Custom map and exploration experiment

```
####################
#S......#.........#
#.#####.#.#########
#.#...#...#.....#.#
#.#.#.#####.###.#.#
#...#.....L.#H#...#
###.########.#.#####
#.....#.....#.....#
#.###.#.#####.###.#
#.#D#...#...P...#.#
#.#.#####.#####.#W#
#.................#
####################
```

My map (maps/my_campus.txt, 13 × 19). S is at (1, 1) and W is at (10, 17). **Dead ends:** the biggest is a 20-cell branch right next to S. It goes east along row 1, dips down through (3, 7)-(3, 9), then continues east to (1, 17), right above W, and stops. It looks like a way to W but goes nowhere. Smaller dead ends lead to (3, 17), H, and D. **Loops:** the map has 3 loops: one in the middle through L, one around P, and one around the corridor (7, 13)-(7, 17) next to W. **Route choices:** every route goes down column 1 first. The shortest then goes east along the bottom row and up into W (31 moves). Going past P takes 35 moves, and using the corridor above W takes 37. **Search:** BFS 31 moves / 103 expanded; A* 31 / 80. The dead-end branch tricks A*: its cells look close to W, so A* explores the whole branch before it finds the real route.

Question: How does the exploration rate ε change how fast the robot learns, how costly training is, and the final route it learns? **Prediction:** I expected a higher ε to find the goal faster but to get lower rewards during training, because more random moves would take the robot off the route.

What I did. I changed only ε (0.05, 0.2, 0.5) on the S-W task. Everything else stayed the same as in Part B: α = 0.3, γ = 0.95, 2,000 training episodes, a 200-move limit, no slipping, and seeds 0, 1, 2 with a new empty Q-table each time. That makes 9 training runs. After training, each run was tested for 100 episodes with ε = 0 and no more learning. I compared the settings using the summary files and the training CSVs.

ε	Test success / moves	First goal reached (episode)	Success rate, episodes 1-100	Episodes that hit the limit	Total training moves	Training moves, last 500	Reward, last 500	Best training episode
0.05	100% / 31	27.0 ± 9.5	0.24 ± 0.02	93.7 ± 2.9	85109 ± 455	32.7 ± 0.1	7.3 ± 0.1	31, 31, 31
0.2	100% / 31	21.3 ± 3.2	0.28 ± 0.02	95.7 ± 3.2	96880 ± 237	39.0 ± 0.1	1.0 ± 0.1	31, 31, 31
0.5	100% / 31	32.3 ± 11.0	0.20 ± 0.03	114.0 ± 6.6	146647 ± 1413	65.1 ± 0.4	-25.1 ± 0.3	36, 38, 39

Table 3. My map, S-W task, for each ε (average ± standard deviation over seeds 0-2). The shortest route is 31 moves (return 9). "Best training episode" lists the shortest training episode for each seed.

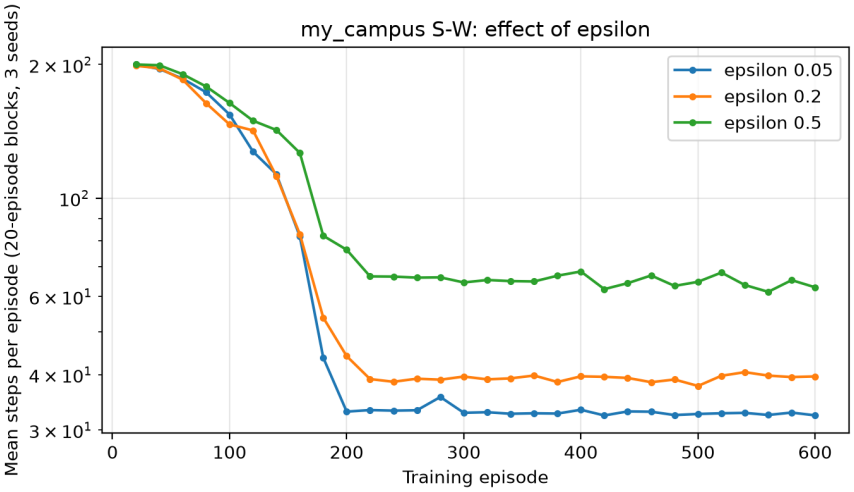


Figure 2. Average moves per training episode, in groups of 20 episodes (3 seeds, log scale), for each ε.

Results. (1) *The final route was the same.* All nine runs reached W 100% of the time in testing, using the shortest 31-move route. (2) *Learning speed barely changed, so the first part of my prediction was wrong.* A higher ε did not find the goal sooner: the first delivery came at episode 27, 21, and 32 on average, and the results varied more between seeds than between settings. All three settings leveled off at about episode 180 to 220 (Figure 2). (3) *Training cost changed a lot, as I predicted.* As ε went up, the total moves in training went from 85k to 97k to 147k, and the average reward late in training dropped from +7.3 to +1.0 to -25.1. With ε = 0.5 the robot never once took the perfect route during training (its best was 36 to 39 moves), and it still failed some episodes late in training (as late as episode 1696). This makes sense: with ε = 0.5, each move is the planned one only 62.5% of the time, so making all 31 moves right in a row almost never happens (about 5 in 10 million). My map was also much harder than the campus map: in the first 100 episodes only 20-28% of episodes reached W (vs 83-91% on the campus map), because of the long route and the big dead end next to S.

What I learned. The surprise was that ε hardly changed how fast the robot learned. I think there are two reasons. First, all Q-values start at 0 and each move costs -1, so actions the robot has not tried look better than ones it has. Even without random moves, it keeps trying new paths, which is how it explored the dead end and the loops early on. Once the dead-end cells had negative values, it stopped going there. Second, Q-learning learns the best route even while it is moving randomly (it is "off-policy"), so extra random moves make training cost more but do not change what it learns. On this small map with no slipping, ε mostly controls how costly training is, and the smallest value, 0.05, worked best. I think ε would matter more with slipping, a much bigger map, or an ε that shrinks over time. Limits: only 3 seeds per setting, one task, and one map.